Function-space sampling in `pygeoinf2`: proposed API

October 23, 2026

This brief lists the proposed changes to the existing API, and two candidate structures for the new sampling package. The reasoning, and the options considered, are in `docs/sampling.tex`. Two rules shape everything below. Every sampler is defined on function space (dimension-robust by construction), and every inference result carries the reference measure its density is defined against. The API changes in Section 1 are needed under either plan.

1. Changes to the existing API

where	kind	change
Measures (`probability/`)		
`AbsolutelyContinuousMeasure`	new	Interface on `ProbabilityMeasure`: `reference`, `log_density` (a `Functional`, $\log d\mu/d\nu$), `log_normalizer` (`Estimate` or `None`), `relative_to(other)`. Built from a reference and a density, never from two arbitrary measures.
`GaussianMeasure.cameron_martin_space()`	new	`MassWeightedSpace(domain, precision, mass_solver=covariance)`; refuses without a precision.
`GaussianMeasure.from_product`	extended	`correlation=` (an $n \times n$ matrix, or a callable with `operator=` for scale-dependent correlation) correlates the latent noise of the factors. `None` keeps today's independent product.
`SymmetricSpace.correlated_measure`	changed	A wrapper over `from_product(correlation=)` that keeps its fused spectral fast path. `sem1d` spaces gain correlated fields.
charted and local references	new	Gaussians equivalent to the prior, each with an r-dimensional `log_density`: the prior with a replaced chart marginal (fitted from samples), and $(C^{-1} + H_r)^{-1}$ from a low-rank Gauss–Newton Hessian. Names open.
Spaces and degenerate methods		
function-space marker	new	Separates discretisations of function spaces (symmetric spaces, `sem1d`, MFEM) from intrinsically finite spaces (Euclidean, data and property spaces).
`log_density`, `mahalanobis_squared`, `grad_log_density`, `log_normalizing_constant`, `credible_set`, `weakened_ellipsoid(power=1)`, mixture `log_density` and `marginal_probabilities`	gated	On a function-space discretisation these refuse unless `unsafe=True`, as D-11 does. Unchanged on finite spaces.
`from_standard_deviation(s)`, `from_samples`, `low_rank_approximation`	unchanged	Not accepted as a prior or reference by any sampler on a function space.
Inference (`inference/`)		
linear Gaussian posteriors	changed	`GaussianEstimator.__call__` returns a subclass of both `GaussianMeasure` and `AbsolutelyContinuousMeasure`: reference the prior, `log_density` $-\Phi$, `log_normalizer` the evidence. Touches D-73.
`LaplaceResult.measure`	changed	The Laplace Gaussian as an `AbsolutelyContinuousMeasure` relative to the prior, with its density computed in the data space.
`ForwardProblem.potential(data)`	new	The likelihood-only potential Φ as a `Functional`, with a temperature.
`MaximumAPosteriori`	extended	Works for priors without a precision, by optimising in whitened coordinates. Open.
Support		
`numerics/`	new	Integrated autocorrelation time, effective sample size and split $\hat{R}$ on arrays.
`testing.check_scheme`	new	Prior invariance at a large step; round-trip reversibility of deterministic blocks; acceptance flat under refinement; slope of bias against step.

where	kind	change
`DECISIONS.md`	changed	Lift D-113's deferral, amend D-7's sampler interface, and record that samplers are function-space only.

2. Plan 1: MCMC as an algorithm

A sampler is a method, as a solver is. A `MarkovKernel` proposes; one driver accepts, adapts in burn-in, and runs chains. This is the `InfGeoInv` structure translated onto spaces. Gibbs sweeps compose kernels, and SMC is a sibling driver. Refreshing a geometric preconditioner at intervals is a burn-in adaptation policy, not a new driver.

```
inference/sampling/
    target.py       Target
    kernels.py      MarkovKernel, Proposal, PCN, GeneralizedPCN, MALA, HMC,
                    GeometricMALA, GeometricHMC, MixtureLabel, Block
    mcmc.py         MCMC
    smc.py          SMC
    results.py      SampledPosterior, WeightedEnsemble
```

```python
class Target:
    """Psi = Phi + log(d nu / d mu_0); the only caller of the forward operator."""
    def __init__(self, problem, data, /, *, reference, temperature=1.0,
                 constraint=None): ...
    def potential(self, state, /) -> float: ...
    def at(self, state, /) -> "TargetPoint": ...      # Phi, D Phi, C grad Phi, cached

@dataclass
class Proposal:
    state: Any
    log_offset: float = 0.0                # beyond Psi(u) - Psi(v)
    potential: float | None = None         # Psi(v), if already evaluated

class MarkovKernel(ABC):
    """Proposes; the driver accepts with log alpha = Psi(u) - Psi(v) + log_offset."""
    uses_derivative: bool = False
    def propose(self, state, target, /, *, rng=None) -> Proposal: ...
    def __matmul__(self, other, /) -> "MarkovKernel": ...   # composition: a Gibbs sweep

class PCN(MarkovKernel):
    def __init__(self, *, step): ...
class GeneralizedPCN(MarkovKernel):                     # step=1: independence sampler
    def __init__(self, reference, /, *, step): ...
class MALA(MarkovKernel):
    def __init__(self, *, step): ...
class HMC(MarkovKernel):
    def __init__(self, *, step, length): ...
class GeometricMALA(MarkovKernel):                      # likewise GeometricHMC
    def __init__(self, *, step, rank, anchor="state"): ...  # or "fixed"
class MixtureLabel(MarkovKernel):                       # Gibbs step on the label
    def __init__(self, mixture, /): ...
class Block(MarkovKernel):                              # one summand of a DirectSum
    def __init__(self, kernel, /, *, summand): ...

class MCMC:
    """Burn-in, adaptation in burn-in only, chains over n_jobs."""
    def __init__(self, problem, prior, /, *, kernel, properties=None,
                 constraint=None): ...
    def __call__(self, data, /, *, steps, chains=1, burn_in=0, adapt=None,
                 start=None, rng=None, n_jobs=None,
                 callback=None) -> "SampledPosterior": ...

class SMC:
    """Reweight, resample and mutate along exp(-beta Phi), with any kernel."""
    def __init__(self, problem, prior, /, *, mutation, particles,
                 schedule="ess"): ...
    def __call__(self, data, /, *, rng=None, n_jobs=None) -> "WeightedEnsemble": ...

class SampledPosterior(AbsolutelyContinuousMeasure):    # reference: the prior
    chains; diagnostics                                 # tau, ESS, R-hat on scalars
    def gaussian_approximation(self, /, *, chart): ...  # charted reference
    def mixture_approximation(self, k, /, *, chart): ...
```

```
    def samples(self, n, /, *, rng=None, n_jobs=None): ...  # fresh short chains

class WeightedEnsemble(AbsolutelyContinuousMeasure):     # log_normalizer: log Z
    particles; weights; effective_sample_size
```

kernel	proposal	`log_offset`	needs
`PCN`	$m_0 + \sqrt{1-\beta^2}(u-m_0) + \beta\xi$	0	Φ
`GeneralizedPCN`	the same about a reference $\nu \sim \mu_0$	0 (ν is in Ψ)	Φ, $\log d\nu/d\mu_0$
`MALA`	pCN plus the drift $-(1-\rho)\,C\nabla\Phi(u)$	drift terms	$D\Phi$
`HMC`	prior-preserving leapfrog	gives $-\Delta H$	$D\Phi$
`GeometricMALA`, `GeometricHMC`	as above, about a local $\nu(u)$	plus the reverse density	Gauss–Newton, rank r
`MixtureLabel`	Gibbs draw of the component	always accepted	r-dimensional chart
`Block`	the inner kernel on one summand	as the inner kernel	

```
posterior = MCMC(problem, prior, kernel=HMC(step=0.2, length=5))(
    data, steps=20_000, chains=4, rng=rng, n_jobs=4)
reference = posterior.gaussian_approximation(chart=chart)
refined = MCMC(problem, prior, kernel=GeneralizedPCN(reference, step=1.0))(
    data, steps=2_000, start=posterior, rng=rng)

gibbs = Block(PCN(step=0.1), summand="level_set") @ Block(MALA(step=0.05), summand="fields")
ensemble = SMC(problem, prior, mutation=PCN(step=0.3), particles=256)(data, rng=rng)
```

3. Plan 2: MCMC as discretised SDE dynamics

A sampler is built from four layers: dynamics, a splitting scheme of exactly solvable flows, a Metropolis correction, and a population. Presets wrap the common combinations. Because a flow may be any invariant kernel (`KernelFlow`), every Plan 1 kernel is a one-letter word, so Plan 2 contains Plan 1.

```
inference/sampling/
    target.py       Target                          # as in Plan 1
    dynamics.py     Dynamics, PreconditionedOU, Overdamped, Underdamped,
                    Hamiltonian, GeneralizedLangevin
    flows.py        Flow, PriorRotation, LikelihoodKick, MomentumRefresh,
                    PriorOU, GradientDrift, KernelFlow
    schemes.py      SplittingScheme
    corrections.py  MetropolizedBlock, MetropolisHastings
    population.py   Chains, Tempering
    results.py      SampledPosterior, WeightedEnsemble   # as in Plan 1
    presets.py      PCN, MALA, HMC, PreconditionedHMC,
                    GeometricMALA, GeometricHMC, SMC
```

```
class Dynamics(ABC):
    """An SDE leaving the posterior invariant; generates its exact flows
    from the reference, so the prior part is never split."""
    state_space: HilbertSpace               # u, or DirectSum (u, v[, s])
    invariant_measure: ProbabilityMeasure
    def flows(self) -> dict[str, "Flow"]: ...

class PreconditionedOU(Dynamics): ...                   # flows: O_u
class Overdamped(Dynamics): ...                         # flows: O_u, B_u
class Underdamped(Dynamics):                            # flows: A', B', O
    def __init__(self, target, /, *, friction): ...
class Hamiltonian(Dynamics): ...                        # A', B', O (full refresh)
class GeneralizedLangevin(Dynamics):                    # auxiliary variables
    def __init__(self, target, /, *, memory_kernel): ...

class Flow(ABC):
    """One exactly solvable piece of the generator."""
    touches: frozenset[str]                 # e.g. {"u"}, {"v"}, {"u", "v"}
    preserves: Literal["reference", "target", None]
    deterministic: bool
    flip_reversible: bool
    def apply(self, state, time, /, *, rng=None) -> "Step": ...

# PriorRotation (A'), LikelihoodKick (B'), MomentumRefresh (O), PriorOU (O_u),
```

```
# GradientDrift (B_u), and KernelFlow(kernel, /, *, touches, preserves) for any
# invariant kernel: Gibbs blocks, mixture labels, structure moves.

class SplittingScheme:
    """A word in application order, e.g. "OBABO" or [("B", .5), ("A", 1), ...]."""
    def __init__(self, dynamics, word, /, *, step, repeat=1,
                 allow_asymmetric=False): ...
    def block(self, word, /) -> "SplittingScheme": ...
    def __call__(self, state, /, *, rng=None) -> "Step": ...

class MetropolizedBlock:          # deterministic, flip-reversible block; flip on reject
    def __init__(self, block, /, *, flip=True): ...
    def within(self, scheme, /) -> "Kernel": ...

class MetropolisHastings:         # stochastic proposal; transition-density ratio
    def __init__(self, proposal, /): ...

# Every kernel and population operator returns
#   Step(state, log_weight, info)
# with log_weight 0 when target-invariant, -dbeta * Phi when tempering.

class Chains:
    """Independent chains (the Wasserstein population)."""
    def __init__(self, count, /): ...
    def __call__(self, kernel, /, *, start, steps, burn_in=0, adapt=None,
                 rng=None, n_jobs=None, callback=None) -> "SampledPosterior": ...

class Tempering:
    """Reweight, resample and mutate along exp(-beta Phi) (Fisher--Rao)."""
    def __init__(self, particles, /, *, mutation, schedule="ess"): ...
    def __call__(self, target, /, *, rng=None, n_jobs=None) -> "WeightedEnsemble": ...
```

preset	dynamics	word	correction	notes
`PCN`	`PreconditionedOU`	O_u	MH on Φ	gradient-free
`MALA`	`Overdamped`	$\mathrm{B}_u\mathrm{O}_u$	MH (density)	
`HMC`	`Hamiltonian`	$\mathrm{O}_\infty(\mathrm{B}'\mathrm{A}'\mathrm{B}')^L$	block	
`PreconditionedHMC`	`Underdamped`	$\mathrm{O}_\gamma(\mathrm{B}'\mathrm{A}'\mathrm{B}')^L$	block, flip	partial refresh
`GeometricMALA`, `GeometricHMC`	as above	w.r.t. local $\nu(u)$	+ reverse density	Gauss–Newton, rank r
`SMC`	any	`Tempering`	log-weights	gives $\log Z$

```
posterior = HMC(problem, prior, step=0.2, length=5)(data, steps=20_000, rng=rng, n_jobs=4)
reference = posterior.gaussian_approximation(chart=chart)
ensemble = SMC(problem, prior, mutation=PCN(step=0.3))(data, particles=256, rng=rng)

# the same machinery, assembled by hand
dynamics = Underdamped(Target(problem, data, reference=prior), friction=1.0)
scheme = SplittingScheme(dynamics, "OBABO", step=0.2)
kernel = MetropolizedBlock(scheme.block("BAB")).within(scheme)
posterior = Chains(4)(kernel, start=prior, steps=20_000, rng=rng, n_jobs=4)
```

4. Order of work

Common to both plans:

1. The function-space marker and the gated methods.
2. Correlated fields through `from_product(correlation=)`.
3. `AbsolutelyContinuousMeasure`, `cameron_martin_space()`, `ForwardProblem.potential`, and the linear return types.

Then, by plan:

	Plan 1	Plan 2
4	`Target`, `MCMC` and the `PCN` kernel, as the benchmark	`Target`, dynamics, flows, schemes, corrections and `Chains`, with the `PCN` preset as the benchmark
5	`MALA` and `HMC` kernels, benchmarked	`MALA`, `HMC` and `PreconditionedHMC` presets, benchmarked
6	charted and local references; geometric kernels and the `SMC` driver, with examples	charted and local references; `KernelFlow`, the geometric presets and `SMC`, with examples

5. Open points

- Which plan. The report recommends Plan 2, staged so that steps 4–5 deliver what Plan 1 would.
- Names and placement, including the reference-construction names.
- One time-step convention, from which β, δ and ε are derived.
- Whether unadjusted schemes are offered on function spaces at all (Plan 2).
- Every-step geometric preconditioning (two Hessians per step), or only fixed and burn-in-adapted references.
- The MAP point for priors without a precision.